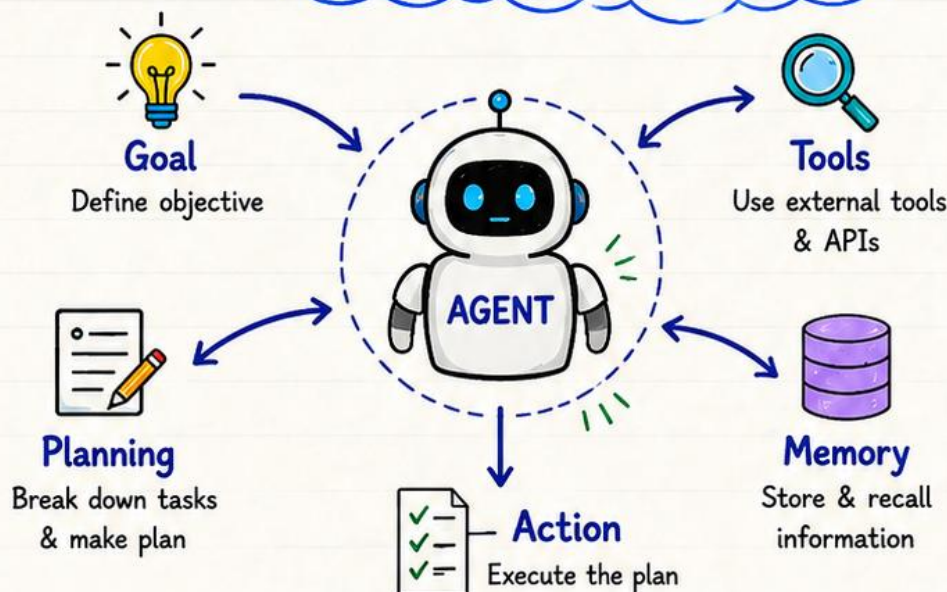




TheAgenticAIEngineer

MACHINE LEARNING & DEEP LEARNING (COMPLETE NOTES)

For Agentic AI



What is Agentic AI?

Agentic AI refers to AI systems that can perceive, reason, plan, and act autonomously to achieve goals using available tools and memory.



SYLLABUS



30 MOST FREQUENTLY
ASKED INTERVIEW QUESTIONS

Benefits for Agentic AI



Smarter Decisions
Learns from data & experience



Goal Oriented
Plans and acts autonomously



Adaptable
Adjusts to new situations



Better Outcomes
Improves efficiency & performance

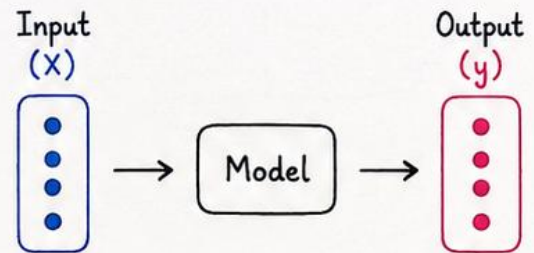


ML Fundamentals



① Supervised Learning

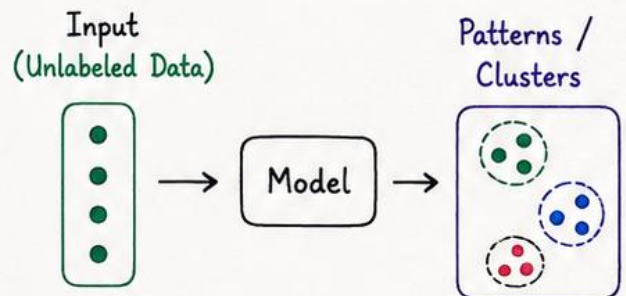
- The model is trained on **labeled data**.
- The goal is to learn a mapping from input (X) to output (y).
- Used for **prediction** and **classification** tasks.
- **Examples:**
Email spam detection, House price prediction, Image classification.



Learn from past data to predict future.

② Unsupervised Learning

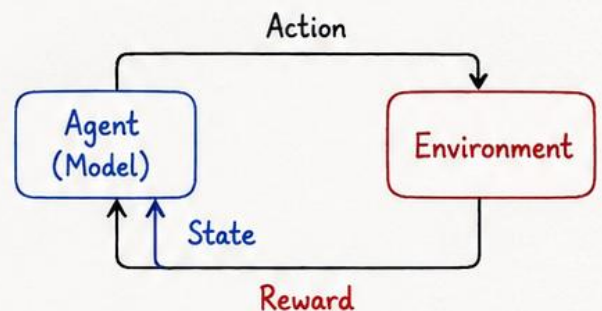
- The model is trained on **unlabeled data**.
- The goal is to find **patterns**, **structure** or **relationships** in the data.
- Used for **clustering**, **dimensionality reduction**, **association**.
- **Examples:**
Customer segmentation, Market basket analysis, Anomaly detection.



Find hidden structure in data.

③ Reinforcement Learning

- The model (**agent**) learns by interacting with an **environment**.
- The goal is to **maximize cumulative reward**.
- Learns through **trial and error**.
- **Examples:**
Game playing (Chess, Atari), Robot navigation, Self-driving cars.



Learn best actions to get maximum reward.



In Short:

Supervised Learning → Learn from **labeled data** to predict.

Unsupervised Learning → Find hidden patterns in **unlabeled data**.

Reinforcement Learning → Learn by interacting and maximizing reward.



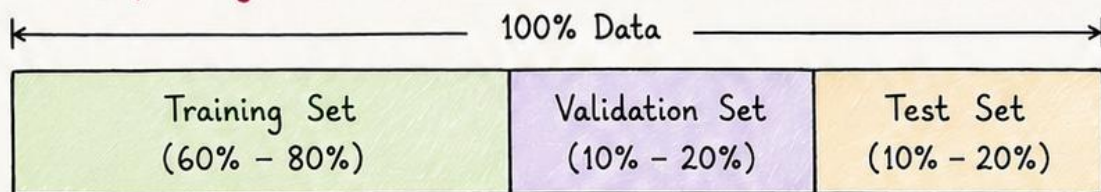
ML Fundamentals



• Training, Validation, Testing •

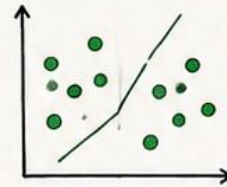
- When we build a machine learning model, we need data to **train** the model and **evaluate** its performance.
- So, we split the dataset into three parts:
Training set, **Validation set**, and **Test set**.

1. Data Splitting



① Training Set

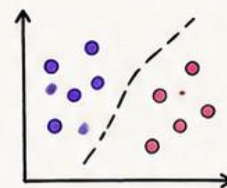
- Used to **train** the machine learning model.
- The model learns patterns and relationships from this data.
- Usually the largest portion of the data.



Model learns from training data.

② Validation Set

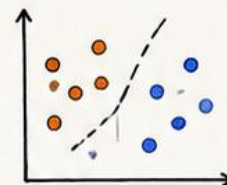
- Used to **tune hyperparameters** and make decisions while training.
- Helps in selecting the **best model**.
- Used to prevent **overfitting**.



Helps choose the **best model** during training.

③ Test Set

- Used to **evaluate** the final model performance.
- The model has never seen this data before.
- Gives an unbiased estimate of how the model will perform on new, **unseen data**.



Tests the **final model** on unseen data.



Key Point:

Train the model → **Validate** and tune → **Test** the final model
→ **Deploy** on real-world data.



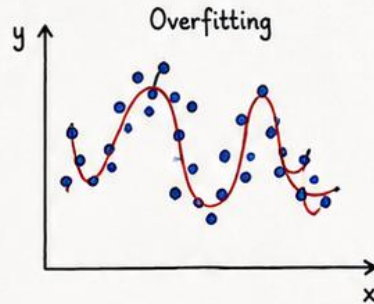
ML Fundamentals



④ Overfitting & Underfitting

• Overfitting :

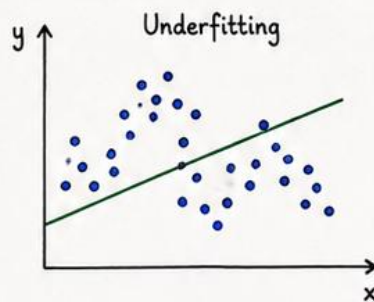
- Model learns the training data too well, including the **noise**.
- Performs very well on training data but **poorly** on new/unseen data.
- **High variance**, Low bias.



Complex model fits training data (noise) too closely.

• Underfitting :

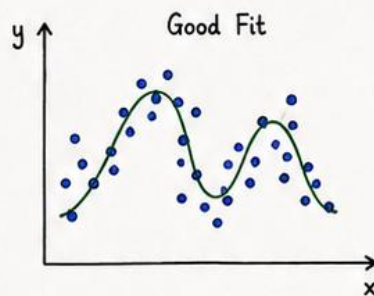
- Model is too simple to capture the underlying pattern.
- Performs **poorly** on both training and new data.
- **High bias**, Low variance.



Simple model cannot capture the pattern (underfits).

• Good Fit (Just Right) :

- Model captures the **pattern** well without fitting the noise.
- Performs well on both training and new data.
- Low bias, Low variance.

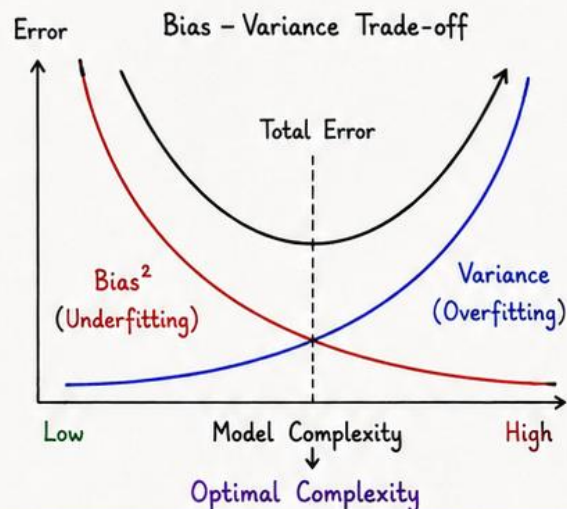


Good balance between bias and variance.

⑤ Bias - Variance

Bias : Error due to wrong assumptions in the model.
(Too simple model)

Variance : Error due to model being too sensitive to small fluctuations in training data.
(Too complex model)



Key Takeaway

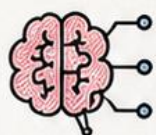
- Low complexity → High bias, Low variance (**Underfitting**)
- High complexity → Low bias, High variance (**Overfitting**)
- Optimal complexity → Best balance between **bias** and **variance**.



Remember :

Goal is to find the **right balance** so the model **generalizes well** to **unseen data**.





ML Algorithms



1. Linear Regression

Linear Regression is a supervised learning algorithm used to predict a continuous value based on one or more input features.

Goal :

Find the best straight line that fits the data points and can be used to make predictions.

Model :

$$y = mx + c$$

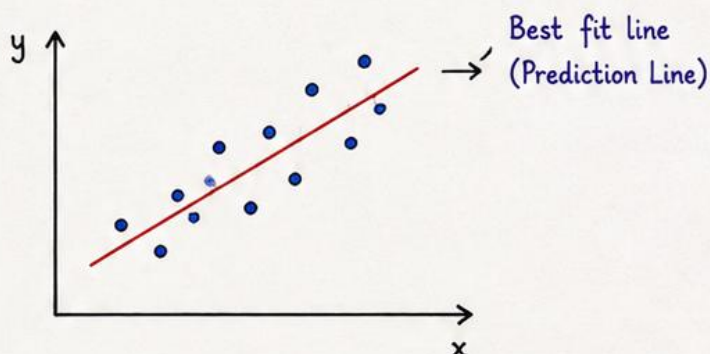
where,

y = Predicted value (output)

x = Input feature

m = Slope (coefficient)

c = Intercept



$m \rightarrow$ tells how much y changes when x changes.

$c \rightarrow$ value of y when $x = 0$ (y - intercept).

How it works ?

- ① Choose initial values of m and c .
 - ② Make predictions ($\hat{y}$).
 - ③ Calculate error (loss).
 - ④ Update m and c to reduce error.
- Repeat until error is minimum.

Types

- Simple Linear Regression $\rightarrow$ One input feature (one x)
- Multiple Linear Regression $\rightarrow$ Two or more input features ($x_1, x_2, \dots, x_n$)

Cost Function

We use Mean Squared Error (MSE) to measure the error.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Where,

y_i = Actual value

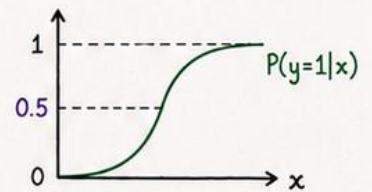
$\hat{y}_i$ = Predicted value

n = Number of data points



In Short : Linear Regression finds the best straight line that minimizes the error between actual values and predicted values.

Logistic Regression



- Logistic Regression is a supervised learning algorithm used for **classification** problems.
- It predicts the **probability** of a class (0 or 1) using the **logistic (sigmoid)** function.

1. Goal

Predict the **probability** that an input belongs to **class 1**.

2. Model

$$P(y = 1|x) = \hat{y} = \frac{1}{1 + e^{-(mx+c)}}$$

where,

x = Input feature

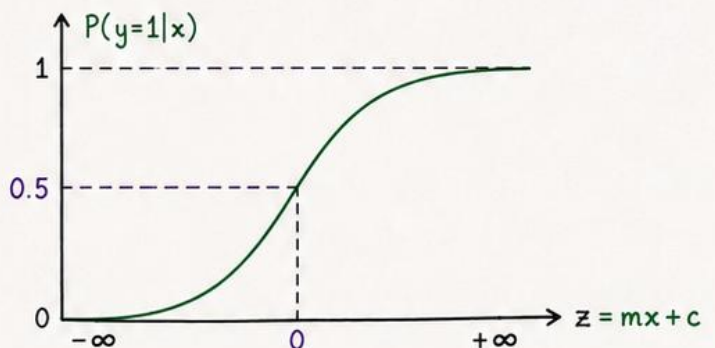
m = Weight (slope)

c = Bias / Intercept

$\hat{y}$ = Predicted probability (0 to 1)

Sigmoid Function

- Converts any real number into a value between 0 and 1.
- S-shaped curve.



3. Decision Rule

If $P(y = 1|x) \geq 0.5 \rightarrow$ Predict **class 1**

If $P(y = 1|x) < 0.5 \rightarrow$ Predict **class 0**

4. Cost Function

We use Log Loss (Binary Cross-Entropy)

$$J = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]$$

where,

y_i = Actual value (0 or 1)

$\hat{y}_i$ = Predicted probability

n = Number of training examples

5. Example

Email Spam Detection

Input Feature: x = Number of words in email

Model:
$$P(y=1|x) = \frac{1}{1 + e^{-(0.08x-4)}}$$

Email (words)	x	$P(y=1 x)$	Prediction
Short email	20	0.05	Not Spam (0)
Medium email	50	0.27	Not Spam (0)
Long email	100	0.73	Spam (1)
Very long email	150	0.92	Spam (1)

As the number of words increases, probability of being spam **increases**.



In Short:

Logistic Regression predicts the **probability** of a binary outcome using the sigmoid function and classifies based on a **threshold** (usually 0.5).

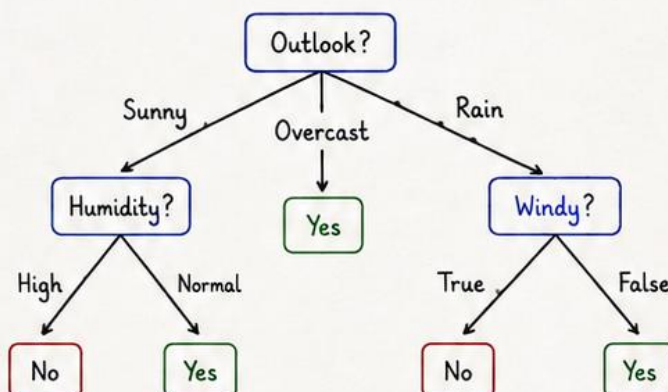


Decision Trees & Random Forest



1. Decision Trees

- Decision Trees are supervised learning algorithms used for both **classification** and **regression**.
- It splits the data based on feature values to make decisions.
- The result is a tree-like model where **internal nodes** are tests, **branches** are outcomes, and **leaf nodes** are predictions.



Example: Predict whether to play tennis based on Outlook, Temperature, Humidity, Windy.

How it works?

- Start with the entire dataset at the root node.
- Choose the best feature to split the data (using Gini Impurity or **Entropy**).
- Split the data into subsets.
- Repeat the process on each subset.
- Stop when a condition is met (pure node, max depth, or minimum samples).

Splitting Criteria

- Gini Impurity (Classification)

$$\text{Gini} = 1 - \sum_{i=1}^c p_i^2$$

Smaller Gini → better split.

- Entropy** (Classification)

$$\text{Entropy} = - \sum_{i=1}^c p_i \log_2 p_i$$

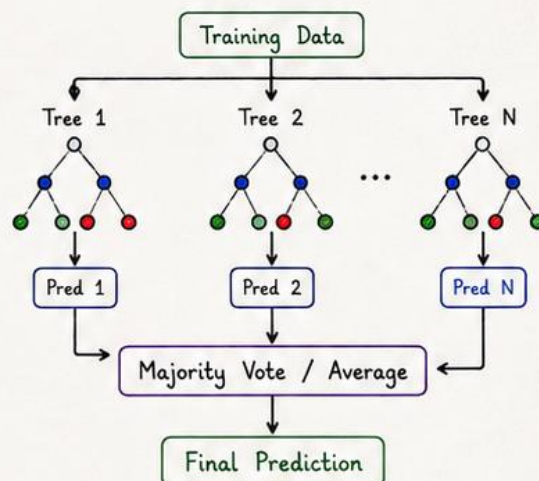
Smaller Entropy → better split.

2. Random Forest

- Random Forest is an **ensemble** of many decision trees.
- It builds multiple trees on **random subsets** of data and features, then combines their predictions.
- It reduces overfitting and improves accuracy.

How it works?

- Draw **bootstrap samples** from training data.
- For each tree, select random features at each split.
- Build a decision tree.
- For classification → **Majority Voting**
For regression → **Average of predictions**
- Final prediction is the combined result.



Example

- Classification: Predict if a customer will buy a product.
- Regression: Predict house prices.

Key Points

- ★ Decision Trees are easy to interpret but can overfit.
- ★ Random Forest reduces overfitting and gives more robust predictions.



XGBoost

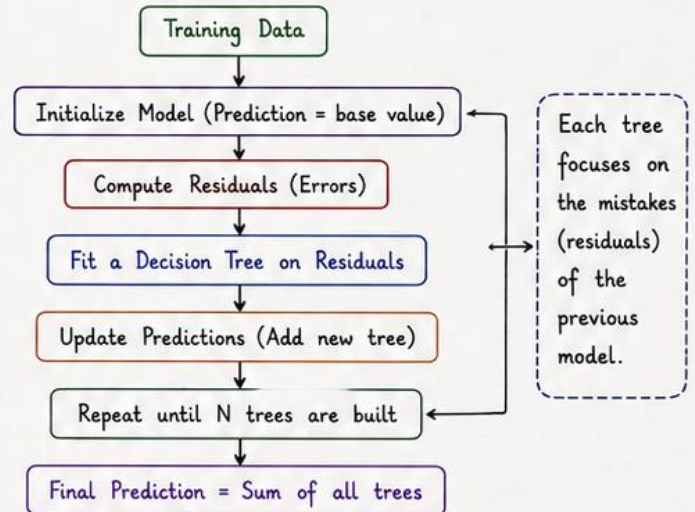


XGBoost (eXtreme Gradient Boosting) is an optimized implementation of Gradient Boosting that is fast, accurate and scalable.

1. Basics

- XGBoost is an ensemble learning method based on Gradient Boosting Decision Trees.
- It builds trees sequentially, where each new tree tries to correct the errors of the previous trees.
- It uses a combination of gradient descent and additive model.
- It has built-in regularization to reduce overfitting and improve generalization.

2. How it works?



3. Objective Function

$$\text{Obj} = \sum_{i=1}^n \ell(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

where,

$\ell(y_i, \hat{y}_i)$ = Loss function (e.g., log loss, MSE)

$\Omega(f_k)$ = Regularization term

K = Number of trees

n = Number of training samples

y_i = Actual value

$\hat{y}_i$ = Predicted value

4. Key Features

- ★ Regularization (L1 & L2) → Controls overfitting
- ★ Shrinkage (learning_rate) → Makes model robust
- ★ Column Subsampling → Uses random subset of features
- ★ Row Subsampling → Uses random subset of data
- ★ Parallel Processing → Faster training
- ★ Handles Missing Values → Automatically learns the best direction for missing values

5. Hyperparameters (Important ones)

Parameter	Description
n_estimators	Number of trees
learning_rate (eta)	Step size shrinkage
max_depth	Maximum depth of a tree
subsample	Fraction of rows used per tree
colsample_bytree	Fraction of columns used per tree
lambda (L2)	L2 regularization term
alpha (L1)	L1 regularization term

6. Use Cases

- ✓ Structured / Tabular data problems
- ✓ Classification (spam detection, churn prediction)
- ✓ Regression (sales forecast, price prediction)
- ✓ Ranking problems (recommendation, search ranking)
- ✓ Competitions (Kaggle, Data Science contests)
- ✓ Any problem where high accuracy and speed are important

Key Takeaway



XGBoost combines the power of Gradient Boosting with regularization, making it one of the most popular and powerful algorithms for real-world ML problems.



SVM & KNN



1. SVM (Support Vector Machine)

SVM is a supervised learning algorithm used for **classification** and **regression**.

It works by finding the **best hyperplane** that separates the classes with **maximum margin**.

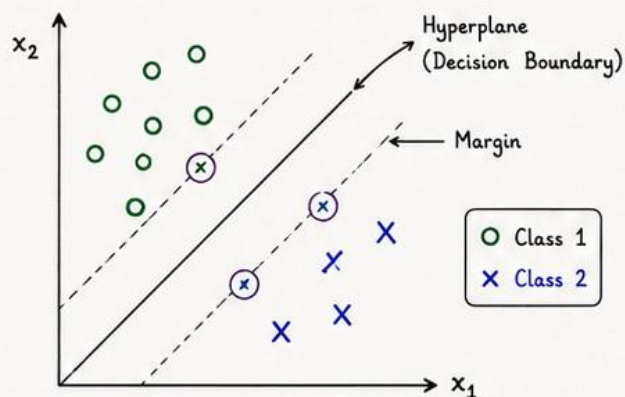
How it works?

- SVM tries to find a hyperplane that separates the data into classes.
- The goal is to maximize the **margin** between the closest points of each class.
- The points closest to the hyperplane are called **Support Vectors**.
- For non-linear data, SVM uses **Kernel Trick** to transform data into higher dimensions.

Use Cases

- ✓ Text classification (spam detection)
- ✓ Image classification
- ✓ Bioinformatics (gene classification)
- ✓ Handwritten digit recognition

Example (Linear SVM)



Key Points

- ★ Effective in high-dimensional spaces.
- ★ Uses kernels: Linear, Polynomial, RBF, Sigmoid.
- ★ Good for small to medium datasets.

2. KNN (K-Nearest Neighbors)

KNN is a simple, instance-based learning algorithm used for **classification** and **regression**.

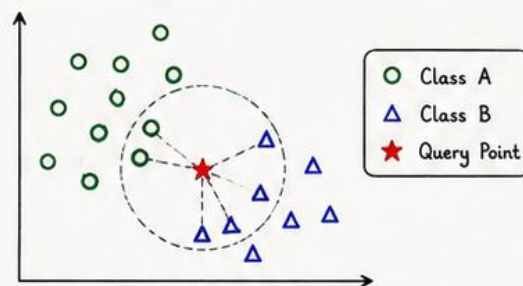
How it works?

- ① Choose the value of K (number of neighbors).
- ② Calculate the distance between the query point and all training points.
- ③ Select the K nearest neighbors.
- ④ For **classification** → take the majority class.
For **regression** → take the average value.

Key Points

- ★ Non-parametric and lazy learning algorithm.
- ★ The choice of K and distance metric is important.
- ★ Sensitive to irrelevant features and scaling.
- ★ Common distance metrics: Euclidean, Manhattan, Minkowski.

Example ($K = 3$)



Here, 2 neighbors are Class A and 1 neighbor is Class B.
→ Predicted class = Class A

Use Cases

- ✓ Recommendation systems
- ✓ Image retrieval
- ✓ Anomaly detection
- ✓ Classification of small datasets



In Short: SVM finds the best boundary (max margin), while KNN looks at nearby neighbors to make a decision.



K-Means & PCA



1. K-Means

K-Means is an **unsupervised clustering** algorithm that partitions data into K clusters by minimizing the within-cluster sum of squares (WCSS).

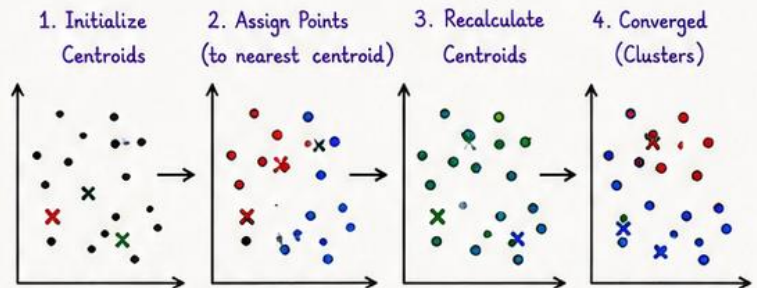
How it works?

- ① Choose the number of clusters K .
- ② Randomly initialize K centroids.
- ③ Assign each data point to the nearest centroid.
- ④ Recalculate centroids as the mean of points in each cluster.
- ⑤ Repeat steps 3-4 until centroids don't change (or max iterations reached).

Key Points

- ★ Simple and easy to understand.
- ★ Works well for spherical (round) clusters.
- ★ Sensitive to initial centroid choice.
- ★ Needs K to be specified in advance.
- ★ Scale the data for better performance.

Example ($K = 3$)



Objective Function

K-Means minimizes the Within-Cluster Sum of Squares (WCSS):

$$WCSS = \sum_{i=1}^K \sum_{x \in C_i} \|x - \mu_i\|^2$$

where, C_i = set of points in cluster i
 μ_i = centroid (mean) of cluster i

Use Cases

- ✓ Customer segmentation
- ✓ Market research
- ✓ Document clustering
- ✓ Image compression
- ✓ Anomaly detection (with clusters)

2. PCA (Principal Component Analysis)

PCA is a **dimensionality reduction** technique that transforms data into a new coordinate system so that the greatest variance lies on the first few principal components.

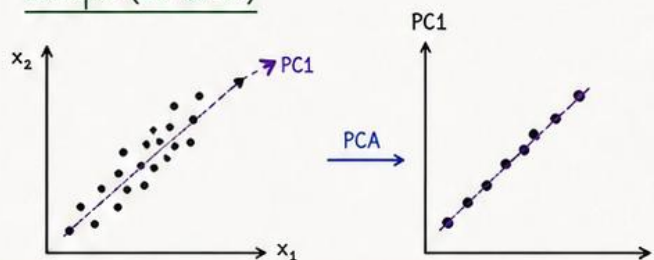
How it works?

- ① Standardize the data.
- ② Compute the covariance matrix.
- ③ Compute eigenvalues and eigenvectors.
- ④ Sort eigenvectors by decreasing eigenvalues.
- ⑤ Choose top k eigenvectors (principal components).
- ⑥ Transform the data to the new k -dimensional space.

Use Cases

- ✓ Dimensionality reduction for high-dimensional data
- ✓ Data visualization (2D/3D plots)
- ✓ Noise reduction
- ✓ Preprocessing for ML models
- ✓ Feature extraction

Example (2D to 1D)



Key Points

- ★ Reduces dimensions while retaining important information.
- ★ First principal component captures maximum variance.
- ★ Orthogonal components (uncorrelated).
- ★ Useful for visualization, noise removal, and faster models.



In Short

K-Means groups data into K clusters by minimizing distance within clusters.

PCA reduces the number of features while keeping the most important information (variance).



Model Evaluation



Model evaluation helps us measure how well our model is performing and how good its predictions are.

Confusion Matrix

1. Accuracy

Accuracy tells us the overall correctness of the model.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Example :

- Total predictions = 100
- TP = 45, TN = 35, FP = 10, FN = 10
- Accuracy = $\frac{45 + 35}{45 + 35 + 10 + 10} = 80\%$

Best use : When classes are balanced.

Limitation : Can be misleading with imbalanced data.

Actual

	Predicted	
	Positive (1)	Negative (0)
Positive (1)	TP (True Positive)	FN (False Negative)
Negative (0)	FP (False Positive)	TN (True Negative)

Where,

TP = Model predicts Positive and it is Positive

TN = Model predicts Negative and it is Negative

FP = Model predicts Positive but it is Negative

FN = Model predicts Negative but it is Positive

2. Precision

Precision tells us how many of the predicted positive cases are actually positive.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Example :

- TP = 45, FP = 10
- Precision = $\frac{45}{45 + 10} = 0.818$ (81.8%)

Best use : When the cost of False Positives is high (e.g., spam detection, fraud detection).

3. Recall (Sensitivity)

Recall tells us how many of the actual positive cases were predicted correctly.

$$\text{Recall} = \frac{TP}{TP + FN}$$

Example :

- TP = 45, FN = 10
- Recall = $\frac{45}{45 + 10} = 0.818$ (81.8%)

Best use : When the cost of False Negatives is high (e.g., disease detection, safety systems).



Key Takeaways

- ★ Accuracy gives overall performance.
- ★ Precision focuses on correctness of positive predictions.
- ★ Recall focuses on capturing all actual positives.
- ★ Use Precision when FP is costly, use Recall when FN is costly.
- ★ For imbalanced datasets, Precision, Recall and F1-Score are more reliable than Accuracy.



Model Evaluation (Part2)



4. F1 - score

F1-score is the harmonic mean of Precision and Recall. It balances both metrics.

$$F1\text{-score} = \frac{2 \times (\text{Precision} \times \text{Recall})}{\text{Precision} + \text{Recall}}$$

Example :

- TP = 45 , FP = 10 , FN = 10
- Precision = $\frac{45}{45 + 10} = 0.818$ (81.8%)
- Recall = $\frac{45}{45 + 10} = 0.818$ (81.8%)
- F1-score = $2 \times \frac{(0.818 \times 0.818)}{0.818 + 0.818} = 0.818$ (81.8%)

Best use : When we want a balance between Precision and Recall.
Useful for imbalanced datasets.

Quick Recap

	Predicted	
	Positive (1)	Negative (0)
Actual Positive (1)	TP (True Positive)	FN (False Negative)
Actual Negative (0)	FP (False Positive)	TN (True Negative)

Where,

TP = Model predicts Positive and it is Positive

TN = Model predicts Negative and it is Negative

FP = Model predicts Positive but it is Negative

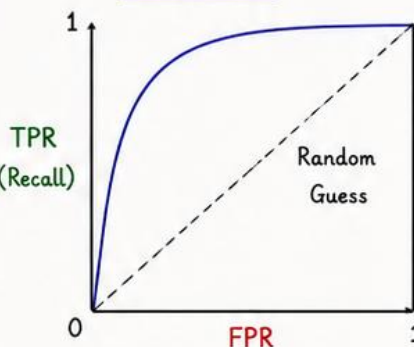
FN = Model predicts Negative but it is Positive

5. ROC-AUC

ROC-AUC evaluates the model's ability to distinguish between classes across all possible thresholds.

- ROC curve plots True Positive Rate (TPR) (Recall) vs False Positive Rate (FPR)
- AUC (Area Under Curve) summarizes the overall performance.

ROC Curve



AUC Score Meaning

- AUC = 1.0 → Perfect model
- AUC = 0.5 → Random guess
- AUC < 0.5 → Worse than random

Key Formulas

$$TPR \text{ (Recall)} = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

Example :

- AUC = 0.95 → Excellent
- AUC = 0.75 → Good
- AUC = 0.50 → No discrimination
- AUC = 0.30 → Poor

Best use :

When class distribution is imbalanced.
Useful for comparing models independent of threshold.



Key Takeaways

- ★ F1-score combines Precision and Recall into a single metric.
- ★ Use F1-score when both Precision and Recall are important.
- ★ ROC-AUC evaluates performance across all thresholds.
- ★ AUC is threshold-independent and ideal for imbalanced datasets.



Model Evaluation (Part3)



6. Cross-validation

Cross-validation is a resampling technique used to evaluate a model's performance on unseen data.

Why use it?

- ★ Provides a more reliable estimate of model performance.
- ★ Reduces overfitting to a single train/test split.

k-Fold Cross-validation

- Split the data into k equal parts (folds).
- Use (k-1) folds for training and 1 fold for testing.
- Repeat k times, each time using a different fold as the test set.
- Average the performance over all k runs.

Example: 5-Fold Cross-validation

Folds →	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Iteration 1	Test	Train	Train	Train	Train
Iteration 2	Train	Test	Train	Train	Train
Iteration 3	Train	Train	Test	Train	Train
Iteration 4	Train	Train	Train	Test	Train
Iteration 5	Train	Train	Train	Train	Test

Common choices of k

- k = 5 → good balance between bias and variance
- k = 10 → lower bias, higher computation

Performance Estimation

$$\text{Final Score} = \frac{(\text{Score}_1 + \text{Score}_2 + \dots + \text{Score}_k)}{k}$$

(Average of k scores)

Best use

- ★ Small to medium datasets
- ★ When data is limited
- ★ To get a stable estimate of model performance

Pros vs. Cons

Pros:

- More reliable than a single split
- Uses all data for both training and validation

Cons:

- Computationally expensive
- Not ideal for very large datasets

7. Confusion Matrix

A confusion matrix is a table used to evaluate the performance of a classification model.

It shows the actual vs, predicted results.

		<u>Predicted</u>	
		Positive (1)	Negative (0)
<u>Actual</u>	Positive (1)	TP (True Positive)	FN (False Negative)
	Negative (0)	FP (False Positive)	TN (True Negative)

Terms

- TP (True Positive): Model predicts Positive and it is actually Positive.
- TN (True Negative): Model predicts Negative and it is actually Negative.
- FP (False Positive): Model predicts Positive but it is actually Negative.
- FN (False Negative): Model predicts Negative but it is actually Positive.

Best use:

- ★ Useful for all classification problems.
- ★ Helps understand types of errors (FP vs FN).

Important Metrics from Confusion Matrix

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall (Sensitivity)} = \frac{TP}{TP + FN}$$

$$\text{F1-score} = \frac{2 \times (\text{Precision} \times \text{Recall})}{\text{Precision} + \text{Recall}}$$



Key Takeaway

- ★ Confusion Matrix gives a complete picture of model performance.
- ★ It is the foundation for many evaluation metrics.



Deep Learning (Part1)



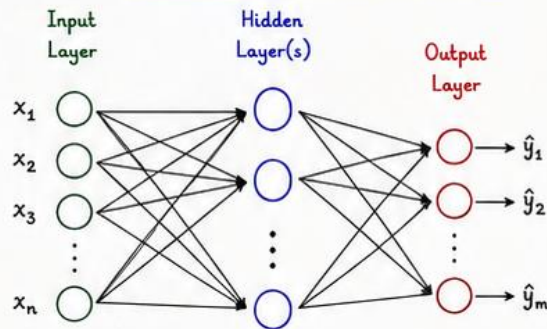
1. Neural Networks

Neural Networks (NNs) are the foundation of deep learning. They are inspired by the human brain and are made up of **interconnected neurons (nodes)** organized in **layers**.

Key Components

- Input Layer : Receives input features
- Hidden Layer(s) : Perform computations
- Output Layer : Produces final prediction
- Weights (W) : Strength of connections
- Bias (b) : Shifts the activation
- Activation Function : Adds non-linearity

Architecture of a Neural Network



How it works?

1. Inputs are multiplied by weights and added with bias.
2. Activation function is applied.
3. Output is passed to the next layer.
4. This continues until the final output is produced.

Key Points

- ★ Learns patterns from data.
- ★ More layers → can learn more complex features.
- ★ Trained using an algorithm called **Backpropagation**.

In Short

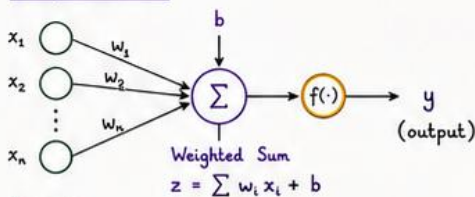


Neural Network =
Layers + Weights +
Activation + Learning

2. Perceptron

Perceptron is the simplest type of Neural Network. It is a single neuron used for binary classification.

Perceptron Model



Example

Inputs: $x_1 = 1, x_2 = 0$

Weights: $w_1 = 0.5, w_2 = -0.5$

Bias: $b = 0.1$

$$z = (0.5 \times 1) + (-0.5 \times 0) + 0.1 = 0.6$$

$y = 1$ (since $z > 0$)

Prediction

$$y = f(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

where,

$$z = \sum_{i=1}^n w_i x_i + b$$

Perceptron Learning Rule

Update weights and bias if prediction is wrong.

$$w_i \leftarrow w_i + \eta (y - \hat{y}) x_i$$

$$b \leftarrow b + \eta (y - \hat{y})$$

where η = learning rate

Key Points

- ★ Linear model.
- ★ Used for binary classification.
- ★ If data is linearly separable, perceptron can find a solution.
- ★ Limitations : Cannot solve non-linearly separable problems (e.g., XOR problem).

Limitations

- ★ Works only for linearly separable data.
- ★ Cannot solve XOR problem.
- ★ Basis for more advanced neural networks.

3. Activation Functions

Activation functions introduce non-linearity into the network, allowing it to learn complex patterns.

Function	Formula	Graph	Range	Use
1. Sigmoid	$\sigma(x) = \frac{1}{1 + e^{-x}}$		(0, 1)	Binary classification (Output layer)
2. Tanh	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$		(-1, 1)	Hidden layers
3. ReLU	$f(x) = \max(0, x)$		$[0, \infty)$	Hidden layers (Most common)
4. Leaky ReLU	$f(x) = \max(\alpha x, x)$ (α is small, e.g., 0.01)		$(-\infty, \infty)$	Hidden layers (Avoids dying ReLU)
5. Softmax	$f_i(x) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$	-	(0, 1) $\sum f_i(x) = 1$	Multi-class classification

Key Takeaways

- ★ Activation functions add **non-linearity**.
- ★ Choice of activation affects model performance.
- ★ ReLU and its variants are widely used in hidden layers.
- ★ Softmax is used in the output layer for multi-class classification.

In Short



Activation Functions =
Brain of Neural Network
(Decides how strongly a neuron should fire!)



Deep Learning (Part2)



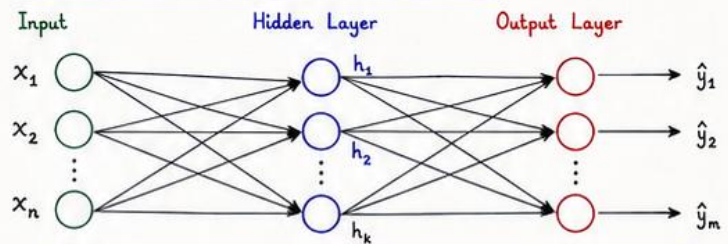
4) Forward & Backpropagation

What is it?

It is the process of training a neural network.

- Forward Propagation → make a prediction.
- Backpropagation → compute error and update weights to reduce the error.

Simple Neural Network (1 Hidden Layer)



1) Forward Propagation

Input moves from left to right.

Step 1: Compute hidden layer

$$z^{(1)} = W^{(1)}x + b^{(1)} \quad \text{where } \sigma \text{ is activation function (e.g., ReLU)}$$

$$a^{(1)} = \sigma(z^{(1)})$$

Step 2: Compute output layer

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)} \quad \text{where } g \text{ is output activation (e.g., Sigmoid) for binary classification.}$$

$$\hat{y} = a^{(2)} = g(z^{(2)})$$

Step 5: Update parameters (using learning rate η)

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}} \quad \text{for } l = 1, 2$$

$$b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial L}{\partial b^{(l)}}$$

2) Backpropagation

Error is computed from right to left.

Step 1: Compute loss

$$L = \text{Loss}(\hat{y}, y)$$

Step 2: Output layer gradients

$$\delta^{(2)} = \frac{\partial L}{\partial z^{(2)}} = \frac{\partial L}{\partial \hat{y}} \odot g'(z^{(2)})$$

Step 3: Hidden layer gradients

$$\delta^{(1)} = (W^{(2)})^T \delta^{(2)} \odot \sigma'(z^{(1)})$$

Step 4: Gradients of weights and biases

$$\frac{\partial L}{\partial W^{(2)}} = \delta^{(2)} (a^{(1)})^T, \quad \frac{\partial L}{\partial b^{(2)}} = \delta^{(2)}$$

$$\frac{\partial L}{\partial W^{(1)}} = \delta^{(1)} x^T, \quad \frac{\partial L}{\partial b^{(1)}} = \delta^{(1)}$$

Key Takeaways

- ★ Forward propagation predicts output.
- ★ Backpropagation computes gradients of loss w.r.t. all parameters.
- ★ Parameters are updated to minimize the loss.



In Short

Forward: Left → Right (prediction)
Backward: Right → Left (learning)
Repeat until loss is minimized!

5) Loss Functions

What is it?

Loss function measures how far the prediction $\hat{y}$ is from the actual target y .

Goal: Minimize the loss during training.

Key Points

- ★ Choose loss based on problem type.
- ★ Lower loss → better model.
- ★ Loss guides the model to learn correct patterns.



In Short

Loss = Measure of Error
↓
Minimize Loss = Better Model

Loss Function	Formula	Used For	Notes
1. Mean Squared Error (MSE)	$L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$	Regression	Penalizes large errors more.
2. Mean Absolute Error (MAE)	$L = \frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i $	Regression	More robust to outliers than MSE.
3. Binary Cross Entropy (BCE)	$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$	Binary Classification	Common for problems with output in (0,1).
4. Categorical Cross Entropy	$L = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$	Multi-class Classification	Generalization of BCE for multiple classes.

Examples

- House price prediction → Use MSE or MAE.
- Email spam (yes/no) → Use BCE.
- Digit classification (0-9) → Use Categorical Cross Entropy.

Remember!

- ★ The whole training process of a neural network is about minimizing the loss function using backpropagation. 😊



Deep Learning (Part3)



6. Gradient Descent

What is it?

Gradient Descent is an optimization algorithm used to minimize the loss function by updating parameters in the direction opposite to the gradient (slope).

Goal: Find parameters (θ) that minimize loss $L(\theta)$.

How it works?

- 1 Start with random parameters θ_0
- 2 Compute gradient of loss w.r.t. parameters
- 3 Update parameters in opposite direction of gradient
- 4 Repeat until loss converges (or stops decreasing)

Update Rule

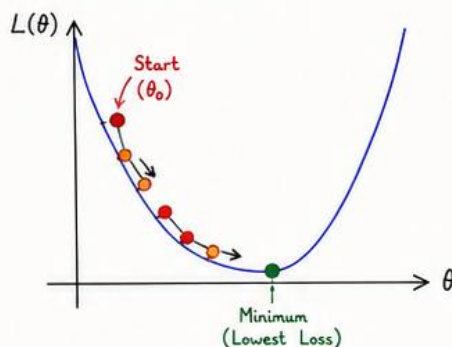
$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t)$$

where,

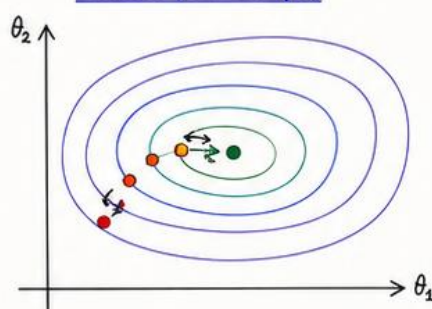
η = learning rate

$\nabla_{\theta} L(\theta_t)$ = gradient of loss at step t

Intuition (1D Example)



Intuition (2D Example)



Key Points

- ★ Moves toward the minimum of the loss function.
- ★ Learning rate (η) controls step size.
- ★ Too large $\eta \rightarrow$ overshoot (diverge).
- ★ Too small $\eta \rightarrow$ slow convergence.

Pros & Cons

Pros:

- ✓ Simple and easy to implement
- ✓ Works for most problems

Cons:

- ✗ Can be slow
- ✗ Sensitive to learning rate
- ✗ May get stuck in local minima (non-convex problems)

In Short



Gradient Descent = take small steps in the direction that decreases loss the most!

7. Optimizers: SGD, Adam, AdamW

Optimizers are advanced versions of Gradient Descent that update parameters more efficiently.

Optimizer	1) SGD (Stochastic Gradient Descent)	2) Adam (Adaptive Moment Estimation)	3) AdamW (Adam with Decoupled Weight Decay)
Idea	Use gradient of a single training example (or mini-batch) to update parameters.	Uses moving average of gradients (momentum) and squared gradients (adaptive learning rate).	Same as Adam but with decoupled weight decay for better regularization.
Update Rule (Parameters)	$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t)$ (With mini-batch)	$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$ $\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$	$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$ $\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta \lambda \theta_t$ (Decoupled weight decay term)
Key Characteristics	• Simple & widely used • Same learning rate for all parameters • No memory of past gradients	• Uses momentum (m_t) • Uses adaptive learning rate (v_t) • Good for sparse gradients	• Same as Adam • Weight decay is decoupled from gradient update (better regularization)
Pros	✓ Easy to implement ✓ Works well with large datasets	✓ Fast convergence ✓ Works well with noisy/sparse data ✓ Less tuning of learning rate	✓ Better generalization (regularization) ✓ Works well for large models (e.g., Transformers)
Cons	✗ Slow convergence ✗ Sensitive to learning rate	✗ More memory (stores m_t and v_t) ✗ May overfit without regularization	✗ Slightly more complex ✗ Needs proper weight decay tuning
Best Used For	• Large scale problems • When simplicity is preferred	• Most deep learning problems • NLP, CV, RNNs, etc.	• Transformer models (BERT, GPT, ViT, ...) • When good generalization is needed

Common Hyperparameters

- Learning rate (η): step size
- β_1 : decay rate for first moment (default 0.9)
- β_2 : decay rate for second moment (default 0.999)
- ϵ : small value to avoid division by zero (default $1e-8$)
- λ (weight decay): regularization strength (AdamW)

Quick Comparison

SGD $\rightarrow$ Simple, slow, needs good η
 Adam $\rightarrow$ Adaptive, fast, general purpose
 AdamW $\rightarrow$ Adaptive + better regularization



AdamW is the preferred optimizer for modern deep learning models!



In Short

- ★ SGD = Basic Gradient Descent (stochastic)
- ★ Adam = Adaptive learning rate + momentum
- ★ AdamW = Adam + better regularization (Decoupled weight decay)

★ Deep Learning (Part4) ★

8) Regularization & Dropout

Why Regularization?

It helps prevent overfitting by discouraging the model from becoming too complex and memorizing the training data.

Common Regularization Techniques

- ★ L1 Regularization (Lasso)
- ★ L2 Regularization (Ridge)
- ★ Dropout

1) L1 & L2 Regularization

Added to the loss function as a penalty on large weights.

Loss Function

$$L_{\text{total}} = L_{\text{loss}} + \lambda \|w\|_1 \quad (\text{L1})$$

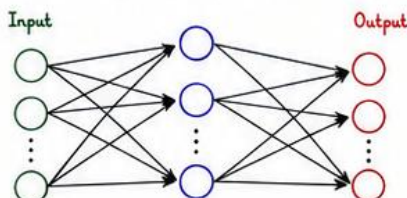
$$L_{\text{total}} = L_{\text{loss}} + \lambda \|w\|_2^2 \quad (\text{L2})$$

where λ = regularization strength

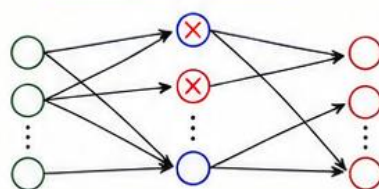
2) Dropout

Randomly drops units (along with their connections) during training with probability p . This prevents co-adaptation of neurons.

Without Dropout (Fully Connected Layer)



With Dropout ($p = 0.5$)



(X) = Dropped unit

Key Points

- ★ Regularization reduces overfitting and improves generalization.
- ★ L1 can lead to sparse weights (some weights become zero).
- ★ L2 keeps weights small but not zero.
- ★ Dropout acts like training many thinned networks and averages them (ensemble effect).
- ★ Dropout is used only during training, not during inference.

In Short



Regularization adds a penalty to the loss. Dropout randomly drops neurons during training. Both help the model generalize better to unseen data.

9) Batch Normalization

What is it?

Batch Normalization (BN) normalizes the inputs of each layer for a mini-batch to have zero mean and unit variance, then scales and shifts them using learnable parameters.

Why use Batch Normalization?

- ★ Accelerates training (faster convergence)
- ★ Allows higher learning rates
- ★ Reduces sensitivity to initialization
- ★ Acts as a regularizer (slight noise)
- ★ Improves overall model performance

How it Works (for one feature)

Given: values $x_1, x_2, \dots, x_m$ in a mini-batch

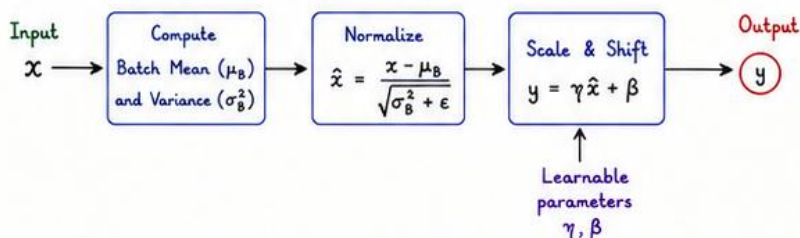
- ① Compute mean: $\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$
- ② Compute variance: $\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$
- ③ Normalize: $\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$
- ④ Scale & shift: $y_i = \gamma \hat{x}_i + \beta$

where γ (scale) and β (shift) are learnable parameters, ϵ is a small constant for stability.

Important Notes

- ✓ Works best with sufficiently large batch sizes.
- ✓ Not always needed in RNNs / sequence models.
- ✓ BN and Dropout can be used together (often in CNNs/MLPs).

BN in a Neural Network Layer



Training vs Inference

During Training:

- Use mini-batch mean and variance.
- Update γ and β using backprop.

During Inference:

- Use running (moving average) of mean and variance:

$$\mu_{\text{running}} = \text{momentum} \cdot \mu_{\text{running}} + (1 - \text{momentum}) \cdot \mu_B$$

$$\sigma_{\text{running}}^2 = \text{momentum} \cdot \sigma_{\text{running}}^2 + (1 - \text{momentum}) \cdot \sigma_B^2$$
- Then normalize using these running stats.

Key Points

- ★ BN is applied after linear/convolution layer and before activation.
- ★ Helps keep the distribution of layer inputs stable.
- ★ Greatly improves training speed and model robustness.

In Short



Batch Normalization normalizes each mini-batch, then scales and shifts using learnable parameters. It makes training faster, more stable and models generalize better.

Analogy



Batch Normalization is like standardizing exam scores in each class before comparing students. Dropout is like randomly ignoring some students while teaching, so no student becomes too dependent on others!



NLP Fundamentals (Part1)



1. Tokenization

What is it?

Tokenization is the process of breaking down text into smaller units called tokens. Tokens can be words, subwords or characters.

Why it is important?

- Converts raw text into a format that models can understand.
- Helps in further NLP tasks like vectorization, embedding, parsing etc.
- Different models use different types of tokenization.

Example

Raw Text:

"NLP is fun! Let's learn together :)"

• **Word Tokenization:**

['NLP', 'is', 'fun', 'Let's', 'learn', 'together', ':)']

• **Subword Tokenization (e.g., BPE):**

['N', 'LP', 'is', 'fun', 'Let', 's', 'learn', 'together', ':)']

• **Character Tokenization:**

['N', 'L', 'P', ' ', 'i', 's', ' ', 'f', 'u', 'n', ' ', '!', ' ', 'L', 'e', 't', ' ', 's', ' ', 'l', 'e', 'a', 'r', 'n', ' ', 't', 'o', 'g', 'e', 't', 'h', 'e', 'r', ' ', ':', ')']

Key Points

- ★ Tokenization is the first step in almost every NLP pipeline.
- ★ Choice of tokenizer affects model performance.
- ★ Subword tokenization helps handle unknown words (out-of-vocabulary problem).

In Short



Tokenization converts raw text into meaningful units (tokens) for machines.

2. Stop words

What is it?

Stop words are very common words in a language that usually carry little meaningful information.

Examples: the, is, at, which, on, and, in, etc.

Why remove them?

- They do not add much value in understanding the meaning.
- Removing them reduces noise.
- Helps in reducing dimensionality and improves model efficiency.

Example

Sentence:

This is a simple example of stop words in NLP."

Tokens:

['This', 'is', 'a', 'simple', 'example', 'of', 'stop', 'words', 'in', 'NLP', '.']

After Removing Stop Words:

['simple', 'example', 'stop', 'words', 'NLP']

Common Stop Words (English)

the, is, are, was, were, be, been, being, and, or, but, if, while, of, in, on, at, by, for, with, etc.

In Short



Stop words are removed to focus on meaningful words that carry information.

3. Stemming

What is it?

Stemming is the process of reducing words to their root or base form by removing affixes (prefixes/suffixes).

Why use Stemming?

- Helps in normalizing words.
- Reduces the number of unique words in the text.
- Improves information retrieval and text analysis.

How it Works?

- ① Takes a word as input.
- ② Removes prefixes and suffixes using predefined rules.
- ③ Returns the root word (stem).

Example

Original Word	Stemmed Word
running	run
jumps	jump
easily	easili
studies	studi
happily	happili
leaves	leav
connected	connect

Example using Porter Stemmer (NLTK)

```
from nltk.stem import PorterStemmer
ps = PorterStemmer()
words = ["running", "jumps", "easily"]
for w in words:
    print(w, "→", ps.stem(w))
```

Output:

```
running → run
jumps → jump
easily → easili
```

Popular Stemmer in NLTK

- Porter Stemmer (most common)
- Snowball Stemmer

Note: Stemming is rule-based and may produce non-dictionary words (e.g., "easili").

In Short



Stemming reduces words to their base form to simplify text and improve NLP processing.



NLP Fundamentals (Part2)



4. TF-IDF

What is it?

TF-IDF (Term Frequency - Inverse Document Frequency) is a statistic that shows how important a word is to a document in a collection.

Why use it?

- Gives higher weight to words that are frequent in a document but rare in all documents.
- Helps in text representation for search, clustering, classification, etc.

Formulas

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

$$IDF(t) = \log \frac{\text{Total number of documents (N)}}{\text{Number of documents containing } t}$$

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

Example

Suppose N = 4 documents.

The word "NLP" appears in 2 documents.

$$\text{Then } IDF(NLP) = \log \frac{4}{2} = \log 2 = 0.301$$

If "NLP" appears 3 times in a document with 100 total words,

$$TF = \frac{3}{100} = 0.03$$

$$TF-IDF = 0.03 \times 0.301 = 0.00903$$

Key Points

- ★ TF measures importance in a document.
- ★ IDF measures rarity in the collection.
- ★ TF-IDF balances both.
- ★ Common words like "the", "is" get low score.
- ★ Very useful for sparse text representations.

In Short



TF-IDF helps highlight words that are important in one document but not common across all documents.

5. Word2Vec / GloVe

What is it?

Word2Vec and GloVe convert words into dense vectors (embeddings) that capture semantic meaning.

Why use it?

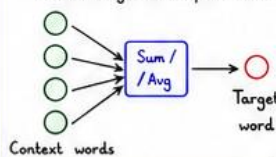
- Captures relationships between words.
- Words with similar meaning have similar vectors.
- Helps models understand context and improve performance.

Word2Vec

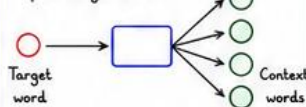
Learns embeddings using prediction objectives on large text corpus.

Two architectures:

- CBOW (Continuous Bag of Words): Predict target word from context.



- Skip-gram: Predict context words from target word.



GloVe (Global Vectors)

Uses global word co-occurrence statistics from the whole corpus.

Builds a word-word co-occurrence matrix and factorizes it to get embeddings.

Captures both local context and global statistics.

Key Points

- ★ Word2Vec: neural network based, learns from context windows.
- ★ GloVe: matrix factorization based, uses co-occurrence counts.
- ★ Both produce dense vectors (e.g., 100, 200, 300 dimensions).
- ★ Useful for similarity, analogy tasks and downstream models.

In Short



Word2Vec and GloVe turn words into numbers (vectors) so that machines can understand meaning and relationships.

6. Sequence models

What is it?

Sequence models are used to process data in sequential order (one element at a time) and capture dependencies.

Why use it?

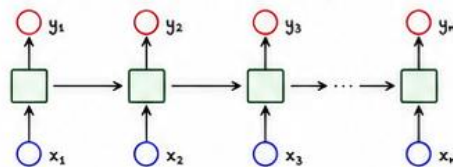
- Language is sequential in nature.
- Helps in tasks like translation, sentiment analysis, summarization, etc.
- Captures context from previous words.

Common Sequence Models

- RNN (Recurrent Neural Network)
- LSTM (Long Short-Term Memory)
- GRU (Gated Recurrent Unit)
- Transformer (e.g., BERT, GPT)

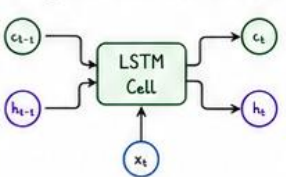
1) RNN (Recurrent Neural Network)

Uses hidden state to remember previous information.



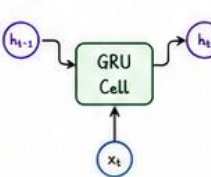
2) LSTM

Uses gates to control information flow.



3) GRU

Simpler version of LSTM.



Key Points

- ★ RNN has vanishing gradient problem.
- ★ LSTM/GRU solve it using gates.
- ★ Transformers use attention instead of recurrence.
- ★ Sequence models power most modern NLP applications.

In Short



Sequence models understand order and context. LSTM/GRU handle long-term dependencies better. Transformers are the latest and most powerful.

Applications

Machine Translation, Text Generation, Sentiment Analysis, Question Answering, Summarization, Chatbots, Speech Recognition, etc.



★ NLP Fundamentals (Part3) ★

7. RNN (Recurrent Neural Network)

What is it?

RNN is a type of neural network that handles sequential data by maintaining a hidden state (memory) that captures information from previous time steps.

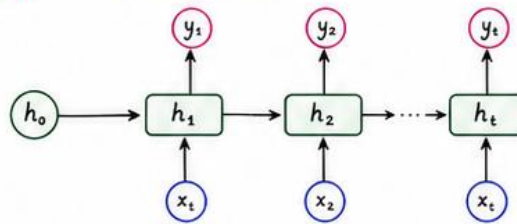
Why use it?

- Works well for sequences (text, speech, time series, etc.)
- Captures context and dependencies.

Common Applications

- ★ Language modeling
- ★ Sentiment analysis
- ★ Machine translation
- ★ Speech recognition

RNN Architecture



At each time step t :

$$h_t = \tanh(W_h x_t + U_h h_{t-1} + b_h)$$

$$y_t = W_y h_t + b_y$$

where:

x_t = input at time t

h_t = hidden state at time t

y_t = output at time t

Key Points

- ★ RNN has loops → information flows through time.
- ★ Suffers from vanishing gradient problem on long sequences.
- ★ LSTM and GRU are improved versions to solve this.
- ★ RNN is simple but effective for short sequences.

In Short



RNN keeps a memory (hidden state) that gets updated at each time step to understand sequences.

8. LSTM (Long Short-Term Memory)

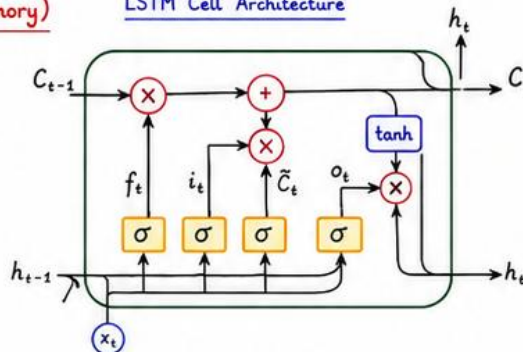
What is it?

LSTM is an advanced RNN that uses gates to control the flow of information and solves the vanishing gradient problem.

Why use it?

- Captures long-term dependencies.
- Remembers important information and forgets irrelevant parts.
- Works well on long sequences.

LSTM Cell Architecture



Equations

$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_c [h_{t-1}, x_t] + b_c)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

Key Points

- ★ LSTM uses gates to control information.
- ★ Cell state acts like a conveyor belt running through the sequence.
- ★ Avoids vanishing gradient problem.
- ★ Best for long sequences and complex patterns.

In Short



LSTM decides what to keep, what to forget and what to output using gates. This helps in learning long-term dependencies.

9. GRU (Gated Recurrent Unit)

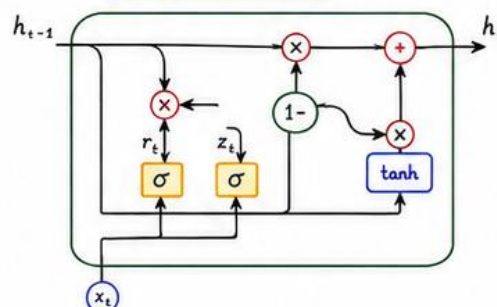
What is it?

GRU is a simplified version of LSTM with fewer gates but similar performance.

Why use it?

- Fewer parameters → faster training.
- Captures long-term dependencies.
- Works well on many NLP tasks.

GRU Cell Architecture



Equations

$$z_t = \sigma(W_z [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Key Points

- ★ GRU has only 2 gates (update and reset).
- ★ No separate cell state like LSTM.
- ★ Faster and simpler to implement.
- ★ Good balance between performance and efficiency.

In Short



GRU is a simpler version of LSTM. It decides what to keep and what to forget using two gates.

Gates in GRU

- ★ Update Gate (z_t): decides how much past to keep.
- ★ Reset Gate (r_t): decides how much past to forget.
- ★ Hidden State (h_t): carries information.

Applications (RNN, LSTM, GRU)

- Language Modeling
- Machine Translation
- Text Summarization
- Sentiment Analysis
- Chatbots